

Neural Network Based Attack on a Masked Implementation of AES

Richard Gilmore, Neil Hanley, Maire O'Neill

Center for Secure Information Technologies,
ECIT, Queen's University Belfast,
Belfast, BT3 9DT, UK.

Email: {rgilmore08, n.hanley, maire.oneill}@qub.ac.uk

Abstract—Masked implementations of cryptographic algorithms are often used in commercial embedded cryptographic devices to increase their resistance to side channel attacks. In this work we show how neural networks can be used to both identify the mask value, and to subsequently identify the secret key value with a single attack trace with high probability. We propose the use of a pre-processing step using principal component analysis (PCA) to significantly increase the success of the attack. We have developed a classifier that can correctly identify the mask for each trace, hence removing the security provided by that mask and reducing the attack to being equivalent to an attack against an unprotected implementation. The attack is performed on the freely available *differential power analysis (DPA) contest data set* to allow our work to be easily reproducible. We show that neural networks allow for a robust and efficient classification in the context of side-channel attacks.

Keywords—SCA, neural network, AES, machine learning, masking.

I. INTRODUCTION

Traditionally, attacks on cryptographic primitives have focused on analysing inputs and outputs of systems to look for mathematical weaknesses in a cipher design. The advent of timing [1] and power [2] attacks however, showed that the underlying implementation of an algorithm in hardware or software can also leak information. This is particularly true in the context of embedded security where an attacker might have direct access to a device. Further research showed that electromagnetic analysis (EMA) could also be used to recover secret key information [3], [4].

Power analysis attacks work on the premise that the instantaneous power consumption of a device is proportional to both the data being manipulated and the operation being performed. In a *non-profiled* scenario, an adversary seeks to use some leakage model $\mathcal{L}$ to estimate the power consumption y for some intermediate value that is a function $\mathcal{F}$ of some known input p and secret s , i.e. $y = \mathcal{L}(\mathcal{F}(p, s))$. As the secret s is unknown, the hypothetical leakage of each element $s^* \in \mathcal{S}$ is calculated, with some statistical distinguisher used to compare y with the actual power consumption x to determine the most likely key $\hat{s}$. Commonly used distinguishers include the difference of means [2], Pearson's linear correlation coefficient [5] and mutual information analysis [6]. While any arbitrary function can be used for $\mathcal{L}$, it is generally based on some engineering intuition of the device under attack. Two models which have been shown to perform well for a wide range of devices are the Hamming weight, and Hamming

distance models [7], which are commonly used when attacking software and hardware devices respectively. A second class of attacks known as *profiling* attacks assume a stronger adversary, in that rather than using some hypothetical power model, an attacker can use a similar or identical device to accurately model the power consumption. This idea was first introduced by Fahn and Pearson in [8], and subsequently extended to Template Attacks (TAs) by Chari *et al.* in [9]. This somewhat stronger, but more restrictive, attack model leads to powerful attacks, and has the advantage of being able to recover secret keys with few or only a single power trace.

The field of side-channel attacks (SCAs) is not purely of academic interest, and there have been multiple examples of attacks on real-world devices such as the KeeLoq remote entry system [10], the bit-stream encryption in Xilinx FPGAs [11], Mifare DESFire contactless payment cards [12], and the YubiKey multi-factor authentication token [13], to name but a few. Due to this real world threat, many cryptographic devices such as smart-cards integrate countermeasures against SCA. Methods such as the randomisation of intermediate values with masking [14], [15], the use of dummy operations [16], or through hiding the data dependent power consumption with the use of secure logic styles [17] can be used to make an attacker's job more difficult. Countermeasures come at a cost however, with increased execution time, power consumption, and area (memory or silicon) requirements, depending on the chosen countermeasures and target platform.

The work in this paper looks at a novel Neural Network (NN) based profiling attack against a masked smart-card implementation of the Advanced Encryption Standard (AES) [18]. The traces used are from version 4 of the DPA contest [19] to allow our results to be easily reproducible. The attack approach proposed in this paper builds on previous research by Lerman *et al.* [20]. A model is first built to recover the mask values. However, rather than performing a non-profiled DPA attack for key recovery, as in [20], in this research, we propose the use of a second model using the same traces used to train the mask model, to recover the key in a single trace with high probability. Also we use a NN rather than an support vector machine (SVM)-based model.

The paper is organised as follows, a synopsis of related work is first provided in Section II, followed by a brief introduction to NNs in Section III. An overview of the data set under consideration is given in Section V, before describing the experimental analysis and results in Section VI. Finally, conclusions are drawn in Section VII.

II. RELATED WORK

Various machine learning methods have been proposed for conducting profiled SCAs. The original TAs [9], or quadratic discriminant analysis (QDA), looked to model the power consumption based on calculating the mean vector $\hat{\mu}^{(i)}$ and noise covariance matrix $\hat{\Sigma}^{(i)}$ for each intermediate value or operation $o^{(i)}$. Different variants of this were also proposed such as using a single covariance matrix (*i.e.* Linear Discriminant Analysis (LDA)) [21], setting non diagonal elements of the covariance matrix to zero (*i.e.* Naïve Bayes), or eliminating the need for a covariance matrix and simply using the euclidean distance as a *reduced* template [7]. Stochastic models (SMs) [22] were also suggested as a linear regression based method for profiling the power consumption.

More recently the use of SVMs [23], [24], random forests (RFs) [24], [25], NNs [26] and self-organising maps (SOMs) [24] have been suggested. Another profiling attack was presented by Sugawara *et al.* [27] based on multivariate regression analysis. This is somewhat similar to the aforementioned SM as it uses linear regression to model expected power consumption values; however, in the classification step correlation power analysis (CPA) (or any other non-profiling distinguisher) is used for key recovery. This was also suggested by Whitnall *et al.* in [28], as well as Lerman *et al.* [20] who use SVMs to target the same data set as used here. Gaussian mixture models (GMM) have also been proposed in [29] in the context of attacking masked implementations to deal with the uncertainty introduced by the unknown masked values.

III. NEURAL NETWORKS

A brief overview is given here, however for a comprehensive description of the topic refer to [30]–[32]. A sample NN diagram is given in Figure 1. It consists of the input layer where each *unit* represents an input feature, multiple hidden layers with differing number of hidden units, and an output layer where each unit represents a class. NNs naturally extend to the multi-class scenario with the addition of extra output units. Each layer except the output also has an extra *bias* unit which acts to shift the activation function curve of a neuron to the left or right.

Every hidden unit and those in the output layer, has an activation function $a_j^{(i)}$ where j is the unit in a given layer, and i is the layer itself. The initial activation units are assigned the input feature values, with subsequent units calculated via some function $\mathcal{G}$, of the weighted sums of the previous layer. This is shown in Equation 1 where the weights are given by Θ , and for clarity the unit subscripts are omitted such that each $a^{(i)}$ term is a vector. While in this work $\mathcal{G}$ is the Sigmoid function, in reality any function can be used although the Sigmoid is widely utilised. The decision of a NN model given a input vector is given by $h_{\Theta}(x^{(l)}) = a^{(l)}$, where l is the number of layers in the model. Each unit of the feed-forward NN outputs a non-linear function of its inputs, which themselves have been mapped non-linearly, allowing complex decision boundaries to be learned [31, ch. 6].

$$a^{(1)} = x^{(i)}, \quad a^{(i)} = \mathcal{G}(\Theta^{(i-1)} a^{(i-1)}) \quad (1)$$

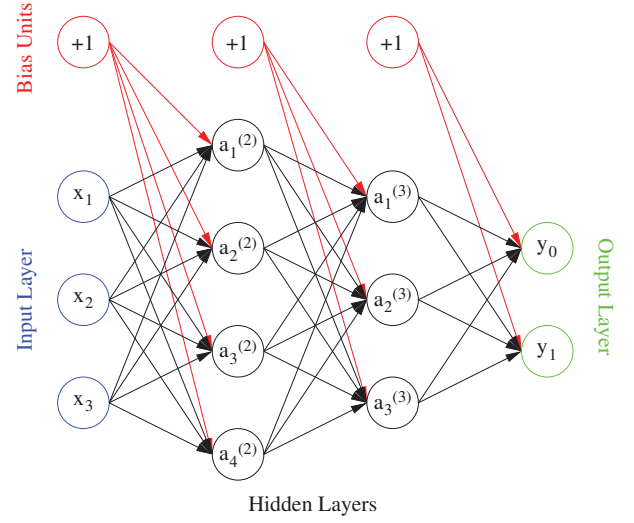


Fig. 1. Artificial Neural Network nodes.

Training a NN model requires learning of the weighting parameters Θ for each layer to minimise the cost function. The *backpropagation* algorithm is commonly used to learn the weights. A number of parameters need to be selected, such as the number of hidden layers, the units per hidden layer and the choice of regularisation parameter λ to prevent over-fitting of data. While parameter choices will be data dependent, a common starting point is to choose a single hidden layer with the number of hidden units set to approximately equal the number of input features. Cross validation can then be used to empirically select the optimal parameters for the data under consideration.

IV. TARGET IMPLEMENTATION

A. Advanced Encryption Standard

In 2001, the Rijndael block cipher by Joan Daemen and Vincent Rijmen, was selected via a public competition by the National Institute of Standards and Technology (NIST) to become the AES [18] as a replacement for the outdated Data Encryption Standard (DES) algorithm. It is a substitution-permutation network (SPN) based iterative block cipher which acts on plaintext blocks of 128-bits and supports significantly larger key sizes than DES, *i.e.* 128-bits, 192-bits or 256-bits. Depending on the key size, the number of rounds is either 10, 12 or 14 respectively. For the work here, the 256-bit key size is examined, although only power traces from the first round are utilised hence the attack is equally viable for the smaller key sizes.

Figure 2 outlines a high-level description of the AES algorithm. First, the plaintext block p is copied into the state variable, which is a 4×4 matrix of bytes. Then, an initial *AddRoundKey* function simply XORs the initial key to the state. This is followed by identical round transformations consisting of the functions; *S-Box*, *ShiftRows*, *MixColumn*, and *AddRoundKey*. The final round skips the *MixColumn* operation to generate the ciphertext.

```

1: procedure AES256(  $p, s$  )
2:    $r \leftarrow p$ 
3:    $r \leftarrow \text{AddRoundKey}(r, s)$ 
4:   for  $i$  in 1 to 14 do
5:      $r \leftarrow S\text{-Box}(r)$ 
6:      $r \leftarrow \text{ShiftRows}(r)$ 
7:     if  $i \neq 14$  then
8:        $r \leftarrow \text{MixColumn}(r)$ 
9:     end if
10:     $r \leftarrow \text{AddRoundKey}(r, s_i)$ 
11:   end for
12:   return  $r$ 
13: end procedure

```

Fig. 2. Advanced Encryption Standard.

B. DPA Contest Traces

The DPA contest [19] provides a standardised set of power consumption measurements to allow for fair comparison of different SCAs by removing the variability due to underlying platform differences, implementation choices, acquisition setup *etc.* The fourth generation of the contest provides side-channel traces acquired from the first round of an AES-256 implementation running on an Atmel ATmega-163 smart-card, using an electromagnetic (EM) probe and a sampling rate of 500 MSs^{-1} . Full acquisition details and source code are available at [19].

The AES implementation is protected by the *Rotating S-Boxes Masking* (RSM) countermeasure. This is a Boolean masking method which randomises the *S-Box* output on every round, providing resistance against 1^{st} and 2^{nd} order DPA attacks. The method of rotating the mask values between rounds allows an efficient implementation both in area and time when compared to other masking methods as only 16 masked *S-Box* tables are required. Each *S-Box* is replaced by a masked version such that the output removes the mask from the input byte and replaces it with a new one, as shown in Figure 3, with the initial offset randomly selected on each execution. Full details can be found in [33].

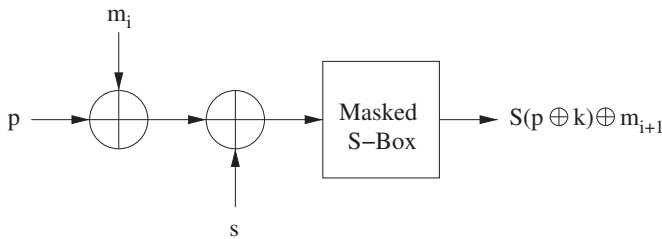


Fig. 3. RSM masking.

V. EXPERIMENTAL ANALYSIS

A. Attack outline

The DPA contest traces only provide the power consumption of the first round of AES-256, hence it is only possible to recover the first 128-bits of the key. However, the attack directly extends to the full 256-bit key when applied to the leakage from round 2 of the algorithm. When performing SCA

it is typical to attack the key a few bits at a time, repeating the attack until the entire key is recovered. Due to the structure of the AES algorithm, the key is attacked byte by byte giving 16 separate multi-class learning problems. In this research work two separate profiling attacks are conducted to recover the secret key. Initially a model is built to determine the masked value, similar to the approach used in [20], [34], then the key is recovered by building a second model on the byte value and using the mask knowledge to remove the randomisation. Novel pre-processing steps are suggested to decrease the error rate. It is assumed the masks are known to an adversary at the profiling stage.

To perform the attack, the trace set is randomly split into *training*, *validation* and *testing* sets. The training set is used for building the NN model, and the validation set then used to select the optimal parameters, *i.e.* the number of features, the number of hidden layers and nodes per layer, and the regularisation parameter λ , by minimising its error rate. As the model is now biased towards optimally classifying the validation set, the testing set is then used to determine the expected error rate on unseen data using these parameters.

1) *Feature Selection*: An important stage of any profiled side-channel attack is to select the relevant features on which to build the model. While the target round or operations might be determined visually by an attacker, many points will have no leakage relevant to the target intermediate value. Many different methods have been proposed for feature selection such as the difference of means (DOM) [9], Student's t-test [35], normalised inter-class variance (NICV) [36] or subspace based methods [37], [38].

In this work as all intermediate values, including masks, are known at the profiling stage, Pearson's linear correlation coefficient, as used in a non-profiled CPA attack, is used to select the features of interest. The 16 mask values are correlated with the traces to determine the points of interest to use to build the NN model, with the features ranked according to the highest absolute correlation. Note that this feature selection step must be performed separately when building NNs for both the recovery of the masks and keys. An example of this feature selection is given in Figure 4 where the correlation of the first byte of the known masked *S-Box* output value is calculated against each point of the power consumption trace. Three distinct sets of peaks occur, where the first is due to the *S-Box* operation itself. The subsequent two groups of peaks are likely due to the masked *S-Box* output value being re-used in the *MixColumn* operation. Regardless, all these peaks contain leakage information that can be used to build a NN model.

2) *Pre-processing steps*: Pre-processing of features can significantly reduce the error rate when performing profiled SCA, and in this research three different methods were tested. As large differences in the ranges of features input into the NN can reduce the impact of changes in variables with small ranges, it is common practice to normalise the input features to a range of $[-1, 1]$. Another common method used to calculate the z-score for each feature as shown in Equation 2. The z-score normalises the inputs, this allows the network to give equal weighting both small and large inputs. Without normalisation the network can be overly influenced by large valued inputs.

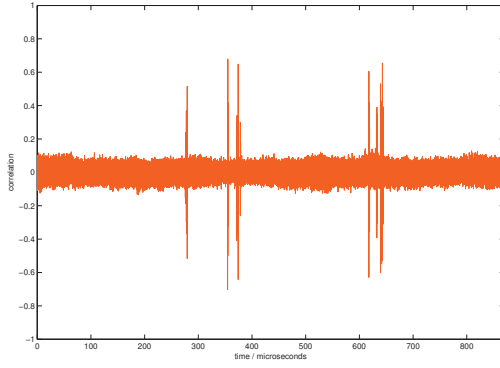


Fig. 4. Correlation between masked *S-Box* output and the power traces.

$$z = \frac{x - \mu}{\sigma} \quad (2)$$

PCA, which orders and orthogonalizes the data, is a widely used technique for dimensionality reduction. Including PCA as a pre-processing step helps reduce the training time of the NN as it reduces the number of features and increases stability of the NN through removal of redundant features [39]. The PCA step is set to discard features in the subspace that contribute less than 2% to the total variance, this was chosen through empirical evidence. Although PCA has previously been used in a SCA for feature selection [37], in this research we propose to use it as a pre-processing step. Note that any pre-processing parameters must be calculated *only* using the training set, and subsequently applied to the validation and testing sets. This differs from the approach followed by Montminy *et al.* in [40] where the testing traces are normalised using parameters estimated from the test set itself. As the objective here is to minimise the error given a single testing trace, it cannot be assumed that multiple traces are available to accurately estimate the normalisation parameters.

B. Mask Removal

In previous work by Lerman *et al.* [20] on the same dataset, the masks were recovered using SVMs prior to performing a CPA attack to recover the key. With a $|\mathcal{K}| = 16$ class SVM classifier, a 90% success rate in the mask recovery stage was achieved, translating to an expected 26.7 traces required in the CPA stage for key recovery. For comparison they also performed the attack with known mask values to see the effect of the misclassifications and 16.2 traces were required. In this research using a NN, and taking a training set of $m = 1500$ and using the same feature selection method, selecting points with highest correlation with offset value in descending order, to select $n = 50$ features as in [20], a success rate of 91.75% was achieved allowing for key recovery in fewer traces. Further extending the model to incorporate a greater number of features, the NN correctly classifies all the mask values from the testing set, thereby removing the security provided by the countermeasure and allowing key recovery in an expected 16.2 traces as if the implementation was unprotected. The validation error learning curve for this

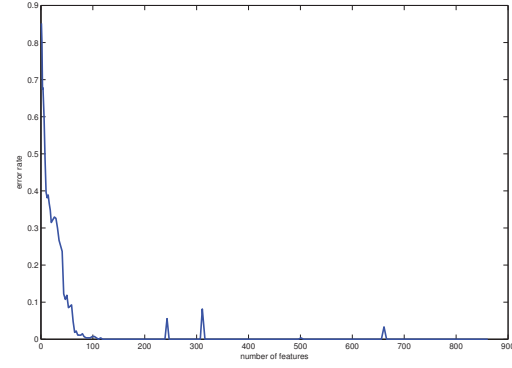


Fig. 5. Error rate for single layer NN where hidden nodes equal number of features and $\lambda = 0.2$

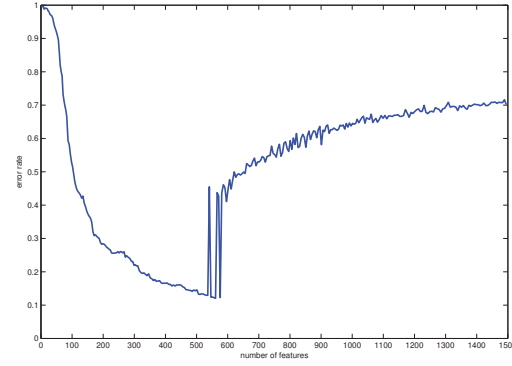


Fig. 6. Error rate for single layer NN where hidden nodes equal number of features and $\lambda = 0.2$, z-score used as the pre-processing step.

experiment is plotted in Figure 5 as a function of the number of retained features, with $n = 119$ features retained when calculating the test error.

C. Key Recovery

With knowledge of the masked values, directly attacking the masked *S-Box* output from Figure 3 is possible with a $|\mathcal{K}| = 256$ multi-class classifier. As all 16 masked values are linked through the rotation, recovering one mask allows the offset to be determined, hence learning the masked *S-Box* output is equivalent to learning a key byte. Using a data set of 10 k traces, with a split of $\{70\%, 15\%, 15\%\}$ for training, cross-validation and testing respectively, the optimal number of features was determined using the validation set as shown in Figure 6. The number of hidden layers and units were determined using the validation set and it was found that a single hidden layer with the number of hidden units set to the number of input features performed well in all cases. Likewise, the regularisation parameter was also learned by minimising the error of the validation set. Training without a regularisation value leads to over-fitting of the training set and poor performance on unseen data. Values in the range of $0.1 \leq \lambda \leq 0.7$ generally gave similar results, with very high regularisation values increasing the training time.

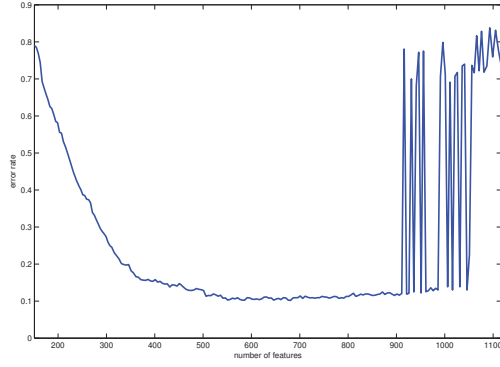


Fig. 7. Error rate for single layer NN where hidden nodes equal number of features and $\lambda = 0.2$, PCA used as the pre-processing step.

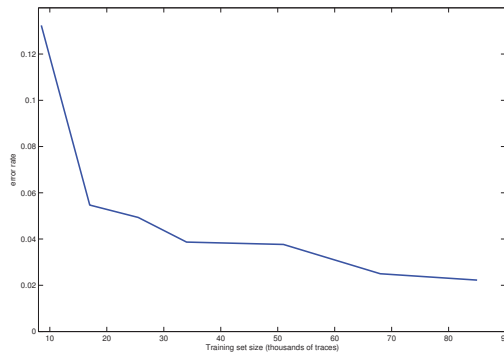


Fig. 8. Error rate versus training set size for single layer NN where hidden nodes equal number of features and regularisation = 0.2

In this case of Figure 6 the features were normalised, and around $n = 500$ features are found to minimise the validation error. The error on unseen data using this number of features is calculated to be 0.128 from the testing set, however the classifier quickly becomes unstable after this point as extra features are included. The advantage of applying PCA as a pre-processing step is evident in Figure 7, as well as producing a small decrease in the test error to 0.116, the NN classifier is now more robust relative to the number of features retained.

The impact on the training set size when using the parameters as determined from the previous experiments ($n = 500$, PCA preprocessing, and $\lambda = 0.6$) is shown in Figure 8. Using the full available data set, $m = 85$ k traces were used to train the NN resulting in a test error rate of only 0.0223. Reducing the error value to $\approx 2\%$ allows the classifier to return the key value in a single trace with high probability. The training time on a classifier of this size however is significant, taking ≈ 4 hours to learn the model compared to ≈ 4 minutes with a training set of $m = 7$ k.

The error rate for all 16 *S-Boxes* classified individually is reported in Table I. It is interesting to note that the different *S-Boxes* are not classified equally hence the results for a single key byte cannot be arbitrarily extrapolated to the key as a whole. The results in Table I were obtained using a training

<i>S-Box</i>	1	2	3	4	5	6	7	8
Error	0.128	0.065	0.061	0.130	0.156	0.046	0.040	0.192
<i>S-Box</i>	9	10	11	12	13	14	15	16
Error	0.154	0.112	0.188	0.196	0.160	0.077	0.150	0.110

TABLE I. NEURAL NETWORK RESULTS.

set of $m = 7$ k. It is probable that using a larger training set the error rate values would be reduced as suggested by Figure 8.

VI. CONCLUSION

In this research it is shown that if an adversary has access to the mask values during the profiling stage of an attack, that NNs can be used to efficiently remove the randomisation due to the mask allowing subsequent CPA or the building of a second NN for key recovery in a few or only a single power trace. NNs are shown to return a lower error rate (91.75% vs. 90%) on the same data set than previous work using SVMs when the same feature selection and training set size was used, and recovered the masks of all test samples correctly when extra features were added without increasing the training set size. Different selection parameters were explored when training a NN were explored using a $|\mathcal{K}| = 256$ multi-class NN classifier to recover the masked *S-Box* output directly, showing that the correct selection of parameters and pre-processing steps can significantly improve the performance of the classifiers.

An underlying assumption of this work is that an attacker has access to the random masks that are used on the profiling device under his control. While this assumption is used in current state of the art on profiling attacks against masked implementations [20], [34], it might not be a practical assumption in a real-world scenario. Further work will investigate profiling attacks without the need for this assumption.

REFERENCES

- [1] P. C. Kocher, "Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and other Systems," in *Advances in Cryptology — CRYPTO 1996*, ser. Lecture Notes in Computer Science, N. Kobitz, Ed., vol. 1109. Springer-Verlag, 1996, pp. 104–113.
- [2] P. C. Kocher, J. Jaffe, and B. Jun, "Differential Power Analysis," in *Advances in Cryptology — CRYPTO 1999*, ser. Lecture Notes in Computer Science, M. J. Wiener, Ed., vol. 1666. Springer-Verlag, 1999, pp. 388–397.
- [3] K. Gandolfi, C. Mourtel, and F. Olivier, "Electromagnetic Analysis: Concrete Results," in *Cryptographic Hardware and Embedded Systems — CHES 2001*, ser. Lecture Notes in Computer Science, Çetin Kaya Koç, D. Naccache, and C. Paar, Eds., vol. 2162. Springer-Verlag, 2001, pp. 251–261.
- [4] J.-J. Quisquater and D. Samyde, "ElectroMagnetic Analysis (EMA): Measures and Counter-Measures for Smart Cards," in *Smart Card Programming and Security — E-Smart 2001*, ser. Lecture Notes in Computer Science, I. Attali and T. P. Jensen, Eds., vol. 2140. Springer-Verlag, 2001, pp. 200–210.
- [5] E. Brier, C. Clavier, and F. Olivier, "Correlation Power Analysis with a Leakage Model," in *Cryptographic Hardware and Embedded Systems — CHES 2004*, ser. Lecture Notes in Computer Science, M. Joye and J.-J. Quisquater, Eds., vol. 3156. Springer-Verlag, 2004, pp. 16–29.
- [6] B. Gierlichs, L. Batina, P. Tuyls, and B. Preneel, "Mutual Information Analysis," in *Cryptographic Hardware and Embedded Systems — CHES 2008*, ser. Lecture Notes in Computer Science, E. Oswald and P. Rohatgi, Eds., vol. 5154. Springer-Verlag, 2008, pp. 426–442.
- [7] S. Mangard, E. Oswald, and T. Popp, *Power Analysis Attacks: Revealing the Secrets of Smart Cards*. Springer-Verlag, 2007.

- [8] P. N. Fahn and P. K. Pearson, "IPA: A New Class of Power Attacks," in *Cryptographic Hardware and Embedded Systems — CHES 1999*, ser. Lecture Notes in Computer Science, Çetin Kaya Koç and C. Paar, Eds., vol. 1717. Springer-Verlag, 1999, pp. 173–186.
- [9] S. Chari, J. R. Rao, and P. Rohatgi, "Template Attacks," in *Cryptographic Hardware and Embedded Systems — CHES 2002*, ser. Lecture Notes in Computer Science, B. S. Kaliski, Çetin Kaya Koç, and C. Paar, Eds., vol. 2523. Springer-Verlag, 2003, pp. 13–28.
- [10] T. Eisenbarth, T. Kasper, A. Moradi, C. Paar, M. Salmisazadeh, and M. T. M. Shalmani, "On the Power of Power Analysis in the Real World: A Complete Break of the KeeLoq Code Hopping Scheme," in *Advances in Cryptology — CRYPTO 2008*, ser. Lecture Notes in Computer Science, D. Wagner, Ed., vol. 5157. Springer-Verlag, 2008, pp. 203–220.
- [11] A. Moradi, M. Kasper, and C. Paar, "Black-Box Side-Channel Attacks Highlight the Importance of Countermeasures - An Analysis of the Xilinx Virtex-4 and Virtex-5 Bitstream Encryption Mechanism," in *Topics in Cryptology — CT-RSA 2012*, ser. Lecture Notes in Computer Science, O. Dunkelman, Ed., vol. 7178. Springer-Verlag, 2012, pp. 1–18.
- [12] D. Oswald and C. Paar, "Breaking Mifare DESFire MF3ICD40: Power Analysis and Templates in the Real World," in *Cryptographic Hardware and Embedded Systems — CHES 2011*, ser. Lecture Notes in Computer Science, B. Preneel and T. Takagi, Eds., vol. 6917. Springer-Verlag, 2011, pp. 207–222.
- [13] D. Oswald, B. Richter, and C. Paar, "Side-Channel Attacks on the Yubikey 2 One-Time Password Generator," in *Research in Attacks, Intrusions, and Defenses — RAID 2013*, ser. Lecture Notes in Computer Science, S. J. Stolfo, A. Stavrou, and C. V. Wright, Eds., vol. 8145. Springer-Verlag, 2013, pp. 204–222.
- [14] L. Goubin and J. Patarin, "DES and Differential Power Analysis (The "Duplication" Method)," in *Cryptographic Hardware and Embedded Systems — CHES 1999*, ser. Lecture Notes in Computer Science, Çetin Kaya Koç and C. Paar, Eds., vol. 1717. Springer-Verlag, 1999, pp. 158–172.
- [15] M.-L. Akkar and C. Giraud, "An Implementation of DES and AES, Secure against Some Attacks," in *Cryptographic Hardware and Embedded Systems — CHES 2001*, ser. Lecture Notes in Computer Science, Çetin Kaya Koç, D. Naccache, and C. Paar, Eds., vol. 2162. Springer-Verlag, 2001, pp. 309–318.
- [16] C. Clavier, J.-S. Coron, and N. Dabbous, "Differential Power Analysis in the Presence of Hardware Countermeasures," in *Cryptographic Hardware and Embedded Systems — CHES 2000*, ser. Lecture Notes in Computer Science, C. Paar and Çetin Kaya Koç, Eds., vol. 1965. Springer-Verlag, 2000, pp. 252–263.
- [17] K. Tiri and I. Verbauwhede, "A Logic Level Design Methodology for a Secure DPA Resistant ASIC or FPGA Implementation," in *Design, Automation and Test in Europe Conference and Exposition — DATE 2004*. IEEE Computer Society, 2004, pp. 246–251.
- [18] NIST, "FIPS-197: Advanced Encryption Standard (AES)," National Institute of Standards and Technology publication, 2001, <http://www.nist.gov/publication-portal.cfm>.
- [19] TELECOM ParisTech SEN research group, "DPA Contest," <http://www.dpacontest.org/v4/index.php>, 2013, version 4.
- [20] L. Lerman, S. F. Medeiros, G. Bontempi, and O. Markowitch, "A Machine Learning Approach Against a Masked AES," in *Smart Card Research and Advanced Application Conference — CARDIS 2013*, ser. Lecture Notes in Computer Science, vol. to appear. Springer-Verlag, 2013.
- [21] O. Choudary and M. G. Kuhn, "Efficient Template Attacks," Cryptology ePrint Archive, Report 2013/770, 2013, <http://eprint.iacr.org/>.
- [22] W. Schindler, K. Lemke, and C. Paar, "A Stochastic Model for Differential Side Channel Cryptanalysis," in *Cryptographic Hardware and Embedded Systems — CHES 2005*, ser. Lecture Notes in Computer Science, J. R. Rao and B. Sunar, Eds., vol. 3659. Springer-Verlag, 2005, pp. 30–46.
- [23] G. Hospodar, E. D. Mulder, B. Gierlichs, J. Vandewalle, and I. Verbauwhede, "Least Squares Support Vector Machines for Side-Channel Analysis," in *Constructive Side-Channel Analysis and Secure Design — COSADE 2011*, W. Schindler and S. A. Huss, Eds., 2011.
- [24] L. Lerman, G. Bontempi, and O. Markowitch, "Side channel attack: an approach based on machine learning," in *Constructive Side-Channel Analysis and Secure Design — COSADE 2011*, W. Schindler and S. A. Huss, Eds., 2011, pp. 29–41.
- [25] L. Lerman, G. Bontempi, S. B. Taieb, and O. Markowitch, "A Time Series Approach for Profiling Attack," in *Security, Privacy, and Applied Cryptography Engineering — SPACE 2013*, ser. Lecture Notes in Computer Science, B. Gierlichs, S. Guilley, and D. Mukhopadhyay, Eds., vol. 8204. Springer-Verlag, 2013, pp. 75–94.
- [26] Z. Martinasek and V. Zeman, "Innovative Method of the Power Analysis," *Radioengineering*, vol. 22, no. 2, 2013.
- [27] T. Sugawara, N. Homma, T. Aoki, and A. Satoh, "Profiling attack using multivariate regression analysis," *IEICE Electronics Express*, vol. 7, no. 15, pp. 1139–1144, 2010.
- [28] C. Whittall and E. Oswald, "Profiling DPA: Efficacy and Efficiency Trade-Offs," in *Cryptographic Hardware and Embedded Systems — CHES 2013*, ser. Lecture Notes in Computer Science, G. Bertoni and J.-S. Coron, Eds., vol. 8086. Springer-Verlag, 2013, pp. 37–54.
- [29] K. Lemke-Rust and C. Paar, "Gaussian Mixture Models for Higher-Order Side Channel Analysis," in *Cryptographic Hardware and Embedded Systems — CHES 2007*, ser. Lecture Notes in Computer Science, P. Paillier and I. Verbauwhede, Eds., vol. 4727. Springer-Verlag, 2007, pp. 14–27.
- [30] C. M. Bishop, *Neural Networks for Pattern Recognition*. Oxford University Press, 1995.
- [31] R. O. Duda, P. E. Hart, and D. G. Stork, *Pattern Classification (2nd Edition)*. Wiley-Interscience, 2001.
- [32] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd Edition*. Springer-Verlag, 2009.
- [33] M. Nassar, Y. Souissi, S. Guilley, and J.-L. Danger, "RSM: A small and fast countermeasure for AES, secure against 1st and 2nd-order zero-offset SCAs," in *Design, Automation & Test in Europe Conference and Exposition — DATE 2012*, W. Rosenstiel and L. Thiele, Eds. IEEE Computer Society, 2012, pp. 1173–1178.
- [34] E. Oswald and S. Mangard, "Template Attacks on Masking — Resistance is Futile," in *Topics in Cryptology — CT-RSA 2007*, ser. Lecture Notes in Computer Science, M. Abe, Ed., vol. 4377. Springer-Verlag, 2007, pp. 243–256.
- [35] B. Gierlichs, K. Lemke-Rust, and C. Paar, "Templates vs. Stochastic Methods," in *Cryptographic Hardware and Embedded Systems — CHES 2006*, ser. Lecture Notes in Computer Science, L. Goubin and M. Matsui, Eds., vol. 4249. Springer-Verlag, 2006, pp. 15–29.
- [36] S. Bhasin, J.-L. Danger, S. Guilley, and Z. Najm, "NICV: Normalized Inter-Class Variance for Detection of Side-Channel Leakage," Cryptology ePrint Archive, Report 2013/717, 2013, <http://eprint.iacr.org/>.
- [37] C. Archambeau, E. Peeters, F.-X. Standaert, and J.-J. Quisquater, "Template Attacks in Principal Subspaces," in *Cryptographic Hardware and Embedded Systems — CHES 2006*, ser. Lecture Notes in Computer Science, L. Goubin and M. Matsui, Eds., vol. 4249. Springer-Verlag, 2006, pp. 1–14.
- [38] F.-X. Standaert and C. Archambeau, "Using Subspace-Based Template Attacks to Compare and Combine Power and Electromagnetic Information Leakages," in *Cryptographic Hardware and Embedded Systems — CHES 2008*, ser. Lecture Notes in Computer Science, E. Oswald and P. Rohatgi, Eds., vol. 5154. Springer-Verlag, 2008, pp. 411–425.
- [39] A. Coates, "Demystifying Unsupervised Feature Learning," Ph.D. dissertation, Stanford University, September 2012.
- [40] D. P. Montminy, R. O. Baldwin, M. A. Temple, and E. D. Laspe, "Improving cross-device attacks using zero-mean unit-variance normalization," *Journal of Cryptographic Engineering*, vol. 3, no. 2, pp. 99–110, 2013.